Ecto changesets are used to insert or update data, and they are significantly different than the approach ActiveRecord takes. Changesets really highlight the functional nature of Elixir. Changesets are built over several function calls: start with the source data, cast input parameters to the right format, and validate that the data is in a valid format. Once you have a valid changeset, you use `Repo.insert` or `Repo.update` to persist the data to the database.

Every application needs to query data from a database. Ecto provides some helpful functions like `Repo.get_by` that quickly find data in the database. But you often need to write custom queries with advanced filters, joins, and ordering. The `Ecto.Query` module provides a complete query DSL that feels a bit like SQL. `Ecto.Query` is integrated into the Elixir language, so it feels natural to use. And it knows about your Ecto schemas, so it can verify that fields exist and are in the correct format.

Finally, you created a context module to expose the public functions of our application. Contexts are useful to organize your application into distinct groupings. But don't sweat the details of contexts too much. Make the best decision for your application contexts based on the information you have at the moment, but remember that you can easily change a context over time.

We'll use this chapter's queries and data in the next chapter. We'll create an API and interface for our application using the Phoenix framework. Things will take shape rapidly, so get ready!